

Khai giảng & Đại số tuyến tính cho AI

Module 1 — Toán học trong AI · Buổi 1 / 22

Khóa học hè 2026 — Python & Machine Learning

Teaching Assistant: Trần Huy

Machine Learning & IoT Lab (ML IoT)

Ho Chi Minh City University of Technology (HCMUT)

Phần 1 — Giới thiệu khóa học

30'

- ▶ Mục tiêu, lộ trình, đánh giá & milestone
- ▶ Nội quy lớp học
- ▶ Môi trường & nền tảng học tập

Phần 2 — Đại số tuyến tính cho AI

135'

- ▶ Data Representation & Feature Space
- ▶ Similarity: dot product, cosine, attention
- ▶ Transformation: ma trận & neural network
- ▶ Learning: AI học ma trận W
- ▶ Dimensionality Reduction: PCA

1

Giới thiệu khóa học

Mục tiêu · Lộ trình · Đánh giá · Nội quy · Nền tảng — 30 phút

Sau khi hoàn thành khóa học, học viên có thể:

1. Vận dụng kiến thức toán nền tảng (đại số tuyến tính, xác suất, giải tích) vào các thuật toán AI.
2. Sử dụng thành thạo các thư viện Python cho AI: **NumPy, Pandas, Matplotlib, Scikit-learn, PyTorch**.
3. Xây dựng pipeline Machine Learning hoàn chỉnh cho bài toán thực tế.
4. Hiểu & áp dụng các kiến trúc Deep Learning phổ biến: **MLP, CNN, Transformer**.
5. Triển khai ứng dụng AI hiện đại: **Object Detection, LLM Agent, Multimodal**.

Phương châm đào tạo: từ nền tảng toán học đến trực giác, lập trình và ứng dụng thực tế.

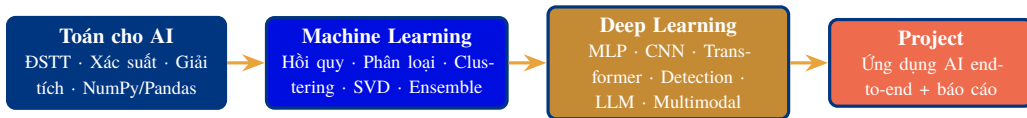
Thông tin chung

- ▶ **Đối tượng:** SV năm 1–2 muốn tìm hiểu AI
- ▶ **Đầu vào:** Python cơ bản & đã học đại số tuyến tính
- ▶ **Thời lượng:** 22 buổi $\times$ 3 tiết = 66 tiết
- ▶ **Thời gian:** 20/6 – 23/8/2026 (11 tuần)
- ▶ **Lịch học:** Thứ Bảy & Chủ Nhật
- ▶ **Hình thức:** trực tiếp tại lớp + máy cá nhân/lab

4 Module

Module	Số buổi	Tuần
1. Toán học trong AI	4	Tuần 1–2
2. Machine Learning	8	Tuần 3–6
3. Deep Learning	6	Tuần 7–9
4. Project cuối khóa	4	Tuần 10–11

Hôm nay: Module 1 — Buổi 1.



Vai trò của nền tảng toán học

Module 1 cung cấp các công cụ toán học được sử dụng xuyên suốt Module 2 và Module 3. Mỗi khái niệm đều được gắn với một ứng dụng cụ thể trong AI.

Tỷ trọng điểm

Tiêu chí	%
Chuyên cần & thái độ	10%
Bài tập thực hành & về nhà	20%
Kiểm tra giữa kỳ	20%
Final Project	50%

Các mốc quan trọng

- ▶ **Bài tập thường xuyên:** 16 bộ (buổi 1–11, 13–17), mỗi buổi có BT tại lớp & về nhà.
- ▶ **Kiểm tra giữa kỳ:** buổi 12 (26/7) — toàn bộ Toán & Machine Learning.
- ▶ **Final Project (milestone chính):** nhóm 4–5 người, xây dựng ứng dụng AI hoàn chỉnh, báo cáo tại buổi 22.

Học viên hoàn thành bài tập theo tiến độ; Lab hỗ trợ qua kênh trực tuyến và đội ngũ mentor.

Chuyên cần & giờ giấc

- ▶ Tham dự đúng giờ; điểm danh đầu mỗi buổi.
- ▶ Chuẩn bị laptop đã cài đặt sẵn môi trường.
- ▶ Giữ điện thoại ở chế độ im lặng, tập trung trong giờ học.

Học tập & ứng xử

- ▶ Chủ động đặt câu hỏi và tham gia thảo luận.
- ▶ Tôn trọng giảng viên và học viên khác.
- ▶ Giữ gìn phòng học và thiết bị chung.

Bài tập & liên chính học thuật

- ▶ Nộp bài đúng hạn qua kênh quy định.
- ▶ Không gian lận, không sao chép mã nguồn; được phép tham khảo nhưng phải tự hiểu và tự trình bày.
- ▶ Trích dẫn nguồn khi sử dụng tài liệu hoặc mã nguồn bên ngoài.

Điều kiện hoàn thành: tham dự tối thiểu **85%** số buổi (vắng tối đa khoảng 3 buổi). Dưới mức này, học viên không đủ điều kiện đánh giá.

Bộ công cụ và phạm vi sử dụng

Công cụ	Vai trò
Python 3.10+	Ngôn ngữ lập trình chính của toàn khóa.
Anaconda	Quản lý package và môi trường ảo, tránh xung đột thư viện.
Jupyter Notebook	Thực thi mã theo từng ô; phù hợp cho học tập và thử nghiệm.
Google Colab	Jupyter trên nền tảng đám mây: không cần cài đặt, có GPU miễn phí.
VSCode	Phát triển dự án quy mô lớn, nhiều tệp.

Phương án dự phòng: nếu việc cài đặt trên máy cá nhân gặp lỗi, học viên sử dụng Google Colab để bắt đầu ngay mà không cần cấu hình.

Discord

Thông báo · trao đổi · thảo luận

- ▶ Kênh liên lạc chính của khóa học.
- ▶ Hỏi đáp với mentor và học viên.
- ▶ Lưu trữ **slide bài giảng & tài liệu**.

GitHub

Mã nguồn & nộp bài tập

- ▶ Mã mẫu và notebook thực hành.
- ▶ **Nộp bài về nhà:** tải repo mẫu → đưa lên repo cá nhân → làm bài và nộp trên đó.
- ▶ Quản lý phiên bản & review mã nguồn project.

Tóm tắt: slide, tài liệu & thảo luận — **Discord**; mã nguồn & nộp bài tập — **GitHub**.

2

Đại số tuyến tính cho AI

Nền tảng toán học cho một hệ thống học máy — 135 phút

Định nghĩa

- ▶ **Vector:** một danh sách số có thứ tự, mô tả một đối tượng qua nhiều đặc trưng.

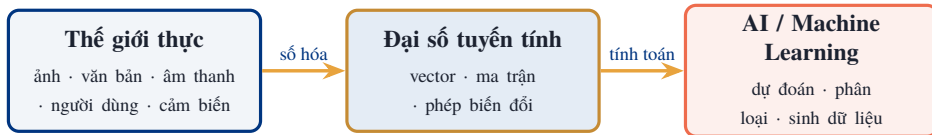
$$\mathbf{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$$

- ▶ **Ma trận:** một bảng số kích thước $m \times n$ — tập hợp nhiều vector, hoặc một phép biến đổi tuyến tính.

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

Vai trò

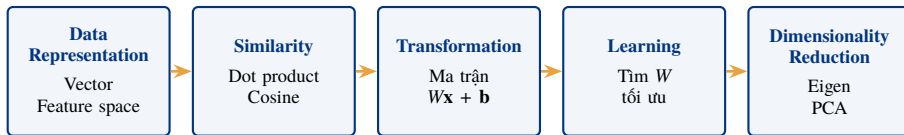
- ▶ Là phương tiện biểu diễn dữ liệu thực tế (ảnh, văn bản, dữ liệu bảng) dưới dạng số mà máy tính xử lý được.
- ▶ Các phép tính trong học máy và học sâu, từ một neuron đến mô hình Transformer, đều được thực hiện trên vector và ma trận.
- ▶ Vector và ma trận do đó là công cụ biểu diễn và tính toán cơ bản trong các mô hình AI.



Từ dữ liệu thực tế đến mô hình AI thông qua đại số tuyến tính.

Hai vai trò cốt lõi

- ▶ **Biểu diễn dữ liệu:** dữ liệu thuộc nhiều dạng khác nhau được biểu diễn thống nhất dưới dạng vector hoặc ma trận.
- ▶ **Nền tảng tính toán:** hầu hết các mô hình, từ hồi quy tuyến tính đến Transformer, đều dựa trên phép nhân ma trận ở phần lõi.



Vai trò xuyên suốt của đại số tuyến tính

Mỗi giai đoạn trong quy trình trên được hiện thực hóa bằng công cụ của đại số tuyến tính: biểu diễn dữ liệu (vector), đo lường tương đồng (dot product, cosine), biến đổi đặc trưng (ma trận), tối ưu hóa tham số (ma trận trọng số) và giảm chiều (eigen, PCA).

Đã học: cơ chế tính toán

Cách nhân hai ma trận
Công thức tích vô hướng
Cách tính ma trận nghịch đảo
Định nghĩa eigenvector

Buổi học này: vai trò trong AI

Một lớp neural network là một phép nhân ma trận
Phép tính được thực hiện nhiều nhất trong mô hình AI
Nghiệm đóng của hồi quy tuyến tính
Trục chính của dữ liệu trong PCA

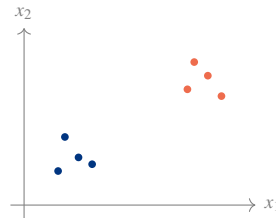
GIAI ĐOẠN 1 · DATA REPRESENTATION

Mỗi đối tượng là một điểm

Một sinh viên được mô tả bằng ba đặc trưng:

$$SV = \begin{bmatrix} \text{GPA} \\ \text{Toán} \\ \text{Python} \end{bmatrix} = \begin{bmatrix} 3.6 \\ 9.0 \\ 8.5 \end{bmatrix}$$

Mỗi sinh viên trở thành **một điểm** trong không gian ba chiều.



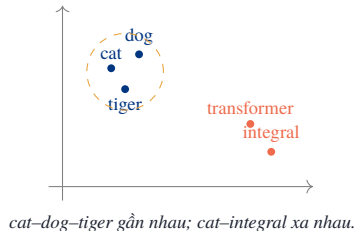
Các đối tượng tương tự hình thành cụm trong không gian đặc trưng.

Trong không gian đặc trưng, các đối tượng có thuộc tính tương tự thường hình thành các cụm (cluster). Nhiều thuật toán học máy khai thác trực tiếp cấu trúc này để thực hiện phân loại hoặc gom cụm dữ liệu.

Mỗi từ là một vector

Đối với mô hình ngôn ngữ, từ vựng không được xử lý dưới dạng ký tự hay chuỗi văn bản; mỗi từ được ánh xạ tới một vector trong không gian biểu diễn nhiều chiều:

$$\text{cat} \rightarrow \begin{bmatrix} 0.13 \\ -0.55 \\ 0.89 \\ \vdots \end{bmatrix} \in \mathbb{R}^{1536}$$



Một tính chất quan trọng của word embedding là **tính chất đại số**: các phép toán trên vector có thể biểu diễn quan hệ ngữ nghĩa ($\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$). Nhờ đó, đo mức liên quan giữa hai từ trở thành đo góc giữa hai vector (cosine similarity).

GIAI ĐOẠN 2 · SIMILARITY

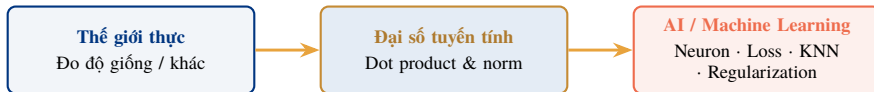
Tích vô hướng & norm

$$\mathbf{w} \cdot \mathbf{x} = \sum_i w_i x_i, \quad \|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$$

- ▶ Dot product: đo mức độ **tương đồng / đồng hướng**.
- ▶ Norm: đo **độ lớn** — khoảng cách, sai số, kích thước trọng số.

Xuất hiện ở đâu

- ▶ Một neuron: $\mathbf{w} \cdot \mathbf{x} + b$.
- ▶ Hàm mất mát: $\text{MSE} = \frac{1}{n} \|\hat{\mathbf{y}} - \mathbf{y}\|_2^2$.
- ▶ KNN, K-Means: phân loại/gom cụm theo khoảng cách.
- ▶ Regularization: ràng buộc $\|\mathbf{w}\|$ chống overfitting.



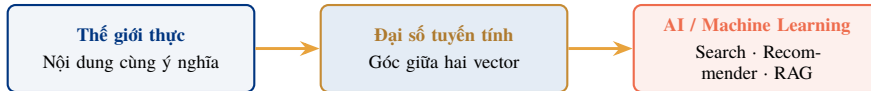
So sánh theo hướng, độc lập với độ lớn

$$\cos \theta = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|} \in [-1, 1]$$

- ▶ Hai nội dung cùng ý nghĩa có embedding gần nhau, $\cos \theta \approx 1$.
- ▶ Không phụ thuộc độ dài văn bản.

Ứng dụng hiện đại

- ▶ Tìm kiếm ngữ nghĩa, gợi ý sản phẩm.
- ▶ **RAG** cho chatbot: truy hồi tài liệu liên quan nhất.
- ▶ Nhận diện khuôn mặt, phát hiện trùng lặp.



Nền tảng của cơ chế Attention

Cơ chế Attention trong Transformer được xây dựng trên phép nhân ma trận và tích vô hướng giữa các vector biểu diễn token.

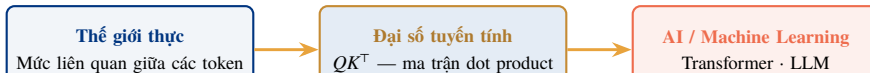
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Scaled dot-product attention sử dụng hệ số $1/\sqrt{d_k}$ để ổn định gradient trong quá trình huấn luyện, tránh hiện tượng giá trị tích vô hướng quá lớn ở không gian chiều cao.

QK^T là gì?

Ma trận chứa tích vô hướng giữa **mọi cặp token** — bảng điểm “token nào liên quan token nào”.

Ma trận QK^T chứa tích vô hướng giữa mọi cặp token, biểu diễn mức độ liên quan giữa chúng; phép tính này được thực hiện với số lượng rất lớn trong mỗi lần suy luận.

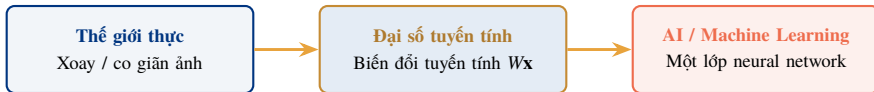
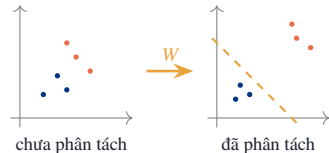


GIAI ĐOẠN 3 · TRANSFORMATION

Nhân ma trận biến đổi không gian đặc trưng

$$y = Wx + b$$

- ▶ W thực hiện phép xoay, co giãn và chiếu không gian.
- ▶ Mỗi lớp của mạng nơ-ron là một phép biến đổi như vậy.
- ▶ Mạng học W sao cho dữ liệu trở nên phân tách được.

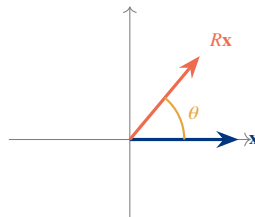


Phép quay góc θ quanh gốc tọa độ

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

- ▶ Áp $R(\theta)$ lên mọi điểm tương đương quay toàn bộ ảnh.
- ▶ Các phép xoay, lật, co giãn ảnh đều là phép nhân ma trận.

Trong AI: *data augmentation* sinh thêm dữ liệu huấn luyện cho CNN (Module 3).





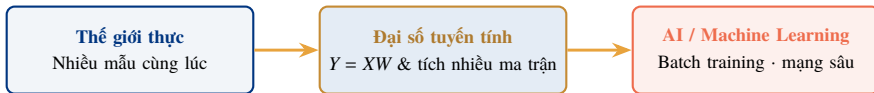
Hai vai trò trong huấn luyện

- ▶ **Xử lý theo batch:** tính đồng thời nhiều mẫu — $Y = XW$, mỗi hàng của X là một mẫu.
- ▶ **Xếp chồng lớp:** mạng nhiều lớp là tích các ma trận $W_3(W_2(W_1x))$, xen kẽ hàm phi tuyến.

Hệ quả

Khối lượng tính toán của AI tập trung gần như hoàn toàn vào phép nhân ma trận.

Lưu ý: $AB \neq BA$ — thứ tự các lớp mang ý nghĩa.





Khối lượng tính toán của AI tập trung ở **phép nhân ma trận quy mô lớn**.

Vai trò của phần cứng song song

- ▶ CPU: tính tuần tự, vài lõi mạnh.
- ▶ GPU: hàng nghìn lõi, thực hiện **Matrix** $\times$ **Matrix** song song cực nhanh.
- ▶ Huấn luyện mô hình = thực hiện phép nhân ma trận hàng tỷ lần.

Hệ quả về phần cứng

Nhu cầu nhân ma trận ở quy mô lớn là động lực chính cho sự phát triển của phần cứng tăng tốc chuyên dụng như **NVIDIA H100, B200** và **Google TPU**.

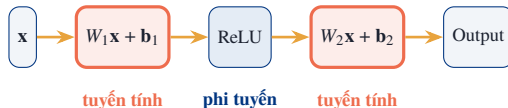
GIAI ĐOẠN 4 · LEARNING



Huấn luyện mô hình là bài toán tối ưu hóa tham số (các ma trận trọng số)

- ▶ Dữ liệu và kiến trúc do con người cung cấp; đại lượng mô hình tự xác định là các ma trận trọng số $W_1, W_2, W_3, \dots$
- ▶ Huấn luyện là quá trình điều chỉnh dần các W để giảm sai số.

Quá trình huấn luyện là bài toán tối ưu hóa các ma trận trọng số nhằm cực tiểu hóa hàm mất mát.



Phần lớn khối lượng tính toán của mạng neural nằm ở các phép biến đổi tuyến tính dạng $W\mathbf{x} + \mathbf{b}$. Điều này giải thích vì sao đại số tuyến tính giữ vai trò trung tâm trong Deep Learning.

Nghiệm trực tiếp, không cần lặp

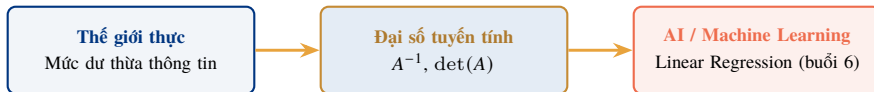
$$\mathbf{w}^* = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (\text{Normal Equation})$$

- ▶ Ma trận nghịch đảo cho phép giải thẳng tham số tối ưu.
- ▶ Đây là một trong số ít mô hình có nghiệm dạng đóng.

Ý nghĩa của định thức

- ▶ $\det \approx 0$ / không khả nghịch $\Rightarrow$ **đa cộng tuyến** (feature dư thừa).
- ▶ Nghiệm bất ổn $\Rightarrow$ cần regularization.

$(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$ là *ma trận giả nghịch đảo* (pseudo-inverse). Đây là nghiệm đóng; với mô hình lớn hơn, ta sẽ tìm nghiệm xấp xỉ bằng **Gradient Descent** (Module 2).

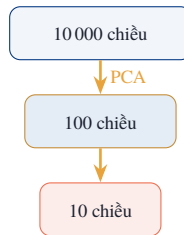


GIAI ĐOẠN 5 · DIMENSIONALITY REDUCTION

Khi số chiều quá lớn

Một ảnh, một văn bản có thể có hàng nghìn đến hàng chục nghìn đặc trưng:

- ▶ Mô hình **khó học** (curse of dimensionality).
- ▶ Nhiều chiều chỉ là **nhhiều**.
- ▶ **Tốn bộ nhớ** và thời gian tính.



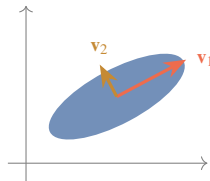
Giảm chiều mà vẫn giữ phần lớn thông tin.

Mục tiêu của giảm chiều là biểu diễn dữ liệu bằng ít chiều hơn trong khi vẫn giữ lại cấu trúc quan trọng nhất.

Hướng được bảo toàn, chỉ thay đổi độ lớn

$$A\mathbf{v} = \lambda\mathbf{v}$$

- ▶ Trên **ma trận hiệp phương sai**, eigenvector chỉ ra hướng dữ liệu biến thiên mạnh nhất.
- ▶ λ lớn = hướng mang nhiều thông tin.
- ▶ PCA giữ vài eigenvector ứng với λ lớn nhất.
- ▶ Trong thực tế, PCA thường được thực hiện qua **Singular Value Decomposition (SVD)** thay vì eigen-decomposition trực tiếp trên ma trận hiệp phương sai, vì SVD ổn định số học hơn với dữ liệu lớn và thưa.



$\mathbf{v}_1$: trục chính, ứng với λ lớn nhất.

Thế giới thực
Hướng biến thiên của dữ liệu

Đại số tuyến tính
Eigenvector / eigenvalue

AI / Machine Learning
PCA · SVD (buổi 10)

Tầng 1 — Trực giác

- ▶ Vector & feature space
- ▶ Similarity (dot, cosine)
- ▶ Phép biến đổi tuyến tính

Nắm vững hôm nay.

Tầng 2 — Trong AI

- ▶ Embedding
- ▶ Neural network học W
- ▶ Attention
- ▶ PCA

Hiểu được vai trò.

Tầng 3 — Chuyên sâu (các buổi sau)

- ▶ SVD — buổi 10
- ▶ Backpropagation — buổi 13
- ▶ Transformer — buổi 15
- ▶ Multimodal — buổi 18

Sẽ học chuyên sâu.

Buổi học hôm nay xây dựng vững Tầng 1 và Tầng 2; Tầng 3 sẽ được trình bày chuyên sâu ở các buổi sau.

Khái niệm hôm nay	Buổi	Nội dung mở rộng
Norm, khoảng cách	5–9	KNN, K-Means, hàm mất mát, regularization
Nghịch đảo, định thức	6	Hồi quy tuyến tính (Normal Equation)
Eigen, hiệp phương sai	10	SVD & PCA, giảm chiều dữ liệu
Ma trận, đạo hàm	13	Backpropagation & chain rule trên ma trận
Dot product, embedding	15	Transformer, self-attention
Cosine, embedding	17–18	LLM / RAG, Multimodal (CLIP)

Các nền tảng trong buổi học này được sử dụng lại và mở rộng trong phần lớn các buổi tiếp theo của khóa.

Mục tiêu

- ▶ Thành thạo các phép toán vector và ma trận cơ bản.
- ▶ Hiểu và sử dụng NumPy một cách hiệu quả.
- ▶ Xây dựng trực giác về cách đại số tuyến tính được triển khai trong code.

Phương pháp: tính tay trước → sau đó viết code NumPy.

Cho hai vector

$$\mathbf{a} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 4 \\ 5 \\ 6 \end{bmatrix}$$

Tính bằng tay: $\mathbf{a} + \mathbf{b}$, $2\mathbf{a}$, $\mathbf{a} \cdot \mathbf{b}$, $\|\mathbf{a}\|_2$.

Lời giải

$$\mathbf{a} + \mathbf{b} = (5, 7, 9)$$

$$2\mathbf{a} = (2, 4, 6)$$

$$\mathbf{a} \cdot \mathbf{b} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32$$

$$\|\mathbf{a}\|_2 = \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14}$$

Cho

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$$

Tính: $A\mathbf{x}$, AB , A^T .

Lời giải

$$A\mathbf{x} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}$$

$$AB = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}, \quad A^T = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

```
import numpy as np

# Tao vector va ma tran
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("a + b =", a + b)
print("2 * a =", 2 * a)
print("a . b =", a @ b)           # dot product
print("A @ B =\n", A @ B)       # nhan ma tran
print("A^T =\n", A.T)           # chuyen vi
```

NumPy cho phép viết code toán học gần với cách viết tay — phong cách vectorized, hiệu quả và nhanh trong AI.



```
import numpy as np

def cosine_similarity(X, Y=None):
    """Cosine similarity giữa các vector (theo hàng)."""
    if Y is None:
        Y = X
    Xn = X / np.linalg.norm(X, axis=1, keepdims=True)    # chuẩn hóa
    Yn = Y / np.linalg.norm(Y, axis=1, keepdims=True)
    return Xn @ Yn.T    # ma trận tương đồng

cat = np.array([[1, 2, 3]])
dog = np.array([[2, 4, 6]])    # cùng hướng
other = np.array([[ -3, 1, 0]])    # khác biệt

print(cosine_similarity(cat, dog)[0, 0])    # ~ 1.0
print(cosine_similarity(cat, other)[0, 0])    # gần 0
```

Đây là phép tính cốt lõi của tìm kiếm ngữ nghĩa, gợi ý sản phẩm và RAG.

```
import numpy as np

theta = np.radians(45)
R = np.array([[np.cos(theta), -np.sin(theta)],      # phép quay
              [np.sin(theta),  np.cos(theta)]]))
S = np.array([[1.5, 0.0],
              [0.0, 0.5]])                          # phép co giãn

square = np.array([[0, 1, 1, 0],
                   [0, 0, 1, 1]])

rotated = R @ square
scaled  = S @ square
print(np.round(R, 3))                               # ma tran quay 45 do
```

Mọi phép xoay, co giãn, lật ảnh đều là phép nhân ma trận.

```
import numpy as np

X = np.array([[1, 2, 3],
              [4, 5, 6]])           # shape (2, 3)

mean = X.mean(axis=0)              # shape (3,)
print(X - mean)                   # broadcasting theo tung cot

v = np.array([10, 20])            # shape (2,)
# X + v -> lỗi: không broadcast được (3 vs 2)
print(X + v.reshape(2, 1))        # reshape thành cot (2, 1)
```

Lời khuyên: luôn kiểm tra X.shape trước và sau mỗi phép tính.



```
import numpy as np

X = np.random.rand(5, 3)
Xc = X - X.mean(axis=0)           # center du lieu

# Cach 1: Eigen decomposition cua ma tran hiệp phương sai
cov = np.cov(Xc, rowvar=False)
eigvals, eigvecs = np.linalg.eig(cov)

# Cach 2: SVD (khuyen nghi dung trong thuc te)
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
print(Vt[:2])                     # 2 thanh phan chinh dau tien
```

Lý do dùng SVD: Ổn định số học hơn, phù hợp với dữ liệu lớn.

```
import numpy as np

sentences = [
    "I love MLIoT Lab",
    "My Teaching Assistant a good boy",
    "I like Artificial Intelligence",
    "Machine Learning is very interesting",
]
E = np.random.rand(len(sentences), 768)      # gia lap embedding

def semantic_search(query, top_k=2):
    q = np.random.rand(1, 768)                # gia lap embedding query
    sims = cosine_similarity(E, q).flatten()
    idx = np.argsort(sims)[-top_k:][::-1]
    return [(sentences[i], round(float(sims[i]), 3)) for i in idx]

print(semantic_search("Tôi ỏmun ọhc AI"))
```

Đây là phiên bản cực kỳ thu gọn của semantic search và RAG.


```
import numpy as np
import time

a = np.random.rand(10000, 300)
b = np.random.rand(10000, 300)

# Cach cham: vong lap Python thuan
start = time.time()
result1 = [np.dot(a[i], b[i]) for i in range(10000)]
print("Python loop:", round(time.time() - start, 3), "giay")

# Cach nhanh: vectorized
start = time.time()
result2 = np.sum(a * b, axis=1)
print("Vectorized :", round(time.time() - start, 3), "giay")
```

Vectorized nhanh hơn rất nhiều — đây là lý do NumPy là công cụ bắt buộc trong AI.

Quy tắc cần ghi nhớ

- ▶ Luôn kiểm tra `.shape` sau mỗi phép toán.
- ▶ Ưu tiên **vectorized operations**, hạn chế vòng lặp Python.
- ▶ Sử dụng `keepdims=True` khi cần giữ chiều.
- ▶ Hiểu rõ quy tắc **broadcasting**.
- ▶ Khi làm việc với dữ liệu lớn → ưu tiên **SVD**.

Thành thạo thao tác NumPy với vector và ma trận là kỹ năng nền tảng cho toàn bộ khóa học.

Sau buổi học, học viên có thể:

- ✓ Cài đặt và sử dụng được Python với Jupyter/Colab.
- ✓ Hiểu được khái niệm feature space và cấu trúc cụm của dữ liệu.
- ✓ Giải thích được vai trò của dot product, cosine, ma trận W và eigen trong AI.
- ✓ Thực hiện được các thao tác vector/ma trận, cosine similarity và phép quay bằng NumPy.
- ✓ Thiết lập được liên hệ giữa bài toán thực tế, đại số tuyến tính và mô hình AI.

Bài tập tại lớp & về nhà: nội dung sẽ được bổ sung; bài tập về nhà nộp qua **GitHub** (repo cá nhân từ repo mẫu).

Nội dung chính

- ▶ Dữ liệu được biểu diễn như các điểm trong không gian đặc trưng.
- ▶ Tích vô hướng và cosine là cơ sở đo mức liên quan, từ KNN đến cơ chế Attention.
- ▶ Mỗi lớp mạng là một phép biến đổi tuyến tính; việc học là tối ưu hóa các ma trận trọng số.
- ▶ PCA/eigen giảm chiều, giữ lại các trục chính của dữ liệu.

Buổi 2 — 21/6

Xác suất thống kê cho AI: xác suất, định lý Bayes, biến ngẫu nhiên & phân phối, đặc trưng thống kê, thực hành trên Iris.

Lưu ý: hoàn tất cài đặt môi trường trước buổi học tiếp theo.

Thank You

Acknowledgements

Cảm ơn các bạn học viên đã tham gia buổi học đầu tiên của khóa **Python & Machine Learning**. Hẹn gặp lại ở Buổi 2 —
Xác suất thống kê cho AI.